

# **Opinión sobre el material de estudio de Arquitectura de Aplicaciones Web**

## **Introducción**

Después de revisar el material de estudio sobre Arquitectura de Aplicaciones Web, considero que fue un material bastante completo y útil para comprender de una manera más amplia cómo se desarrollan las aplicaciones web. Algo que me pareció interesante es que los contenidos no se presentan solamente desde el punto de vista de la programación, sino que muestran diferentes elementos que forman parte de una aplicación y la manera en que estos se relacionan.

El material comienza con los fundamentos del desarrollo web y luego avanza hacia temas más actuales como APIs, arquitecturas modernas, microservicios, nube, automatización, seguridad e Inteligencia Artificial. Esto permite entender que el desarrollo web ha evolucionado y que actualmente un desarrollador necesita conocer mucho más que solamente la creación de páginas.

## **Utilidad del material**

Considero que la documentación fue útil porque me permitió relacionar conocimientos que anteriormente podía ver de manera separada. Por ejemplo, HTML, CSS y JavaScript pueden parecer inicialmente tecnologías independientes, pero al estudiarlas dentro del contexto del desarrollo web se entiende que cumplen diferentes funciones: HTML permite estructurar el contenido, CSS se encarga principalmente de la presentación y JavaScript permite agregar comportamiento e interacción.

También me pareció importante el material de MDN, porque presenta el aprendizaje del desarrollo web de forma progresiva, comenzando desde los fundamentos y avanzando hacia temas necesarios para crear aplicaciones web modernas. Esto ayuda a comprender que antes de utilizar frameworks o tecnologías más avanzadas es necesario tener buenas bases.

## **Arquitectura cliente-servidor y comunicación web**

Uno de los conceptos que considero más importantes fue la arquitectura cliente-servidor. Comprender esta relación permite tener una mejor idea de lo que sucede cuando utilizamos una aplicación web.

El cliente puede realizar una solicitud desde un navegador o una aplicación, mientras que el servidor recibe dicha solicitud, procesa la información y devuelve una respuesta. En este proceso, HTTP tiene un papel fundamental, ya que permite establecer la comunicación entre las diferentes partes.

También pude comprender mejor la importancia de los métodos HTTP. Por ejemplo, GET se utiliza normalmente para obtener información y POST para enviar información, mientras que existen otros métodos destinados a diferentes operaciones. Esto resulta especialmente importante cuando posteriormente se trabaja con APIs.

## **MVC y organización de las aplicaciones**

Otro tema que considero importante es el Modelo Vista Controlador (MVC). Lo que más me llamó la atención de este patrón es la separación de responsabilidades.

La aplicación puede dividirse en Modelo, Vista y Controlador, permitiendo que cada parte tenga una responsabilidad determinada. Esto facilita la organización del proyecto y puede hacer que sea más sencillo realizar cambios, mantener el código y trabajar en equipo.

La documentación de ASP.NET Core muestra precisamente cómo MVC permite separar estas responsabilidades y también incorpora conceptos como routing, validación, inyección de dependencias y Web APIs.

## **Bases de datos, JSON, XML y servicios web**

Considero que otro punto importante es la relación entre las aplicaciones web y las bases de datos. Una aplicación normalmente necesita almacenar y consultar información, por lo que SQL y los sistemas de bases de datos forman parte importante de la arquitectura.

El material también permitió comprender mejor la función de JSON y XML como formatos para representar e intercambiar información. En particular, JSON me parece especialmente importante porque es un formato muy utilizado actualmente para intercambiar datos entre aplicaciones y APIs.

El estudio de SOAP y REST también me permitió entender que existen diferentes formas de lograr la comunicación entre aplicaciones. SOAP utiliza mensajes basados en XML y establece una estructura definida para la comunicación, mientras que REST aprovecha principalmente los principios de la Web y los métodos HTTP.

## **APIs y tecnologías modernas**

Las APIs fueron uno de los temas que considero más relevantes para mi formación. Una API permite que diferentes aplicaciones o componentes puedan comunicarse y utilizar determinadas funcionalidades o información sin necesidad de conocer cómo está construido internamente el otro sistema.

También me pareció interesante conocer REST API y GraphQL como alternativas utilizadas actualmente. El material de GraphQL muestra otra forma de trabajar con APIs, mientras que REST continúa siendo una de las formas más conocidas de construir servicios web.

Otro concepto que me pareció importante fue OpenAPI, porque permite describir una API de una manera estandarizada para que tanto las personas como las herramientas puedan comprender cómo funciona. Esto puede facilitar la documentación, las pruebas y el desarrollo de clientes para una API.

## **Frameworks y diferentes formas de desarrollar**

La cantidad de frameworks y tecnologías presentadas en el material también me permitió entender que no existe una única forma de desarrollar una aplicación web.

React, Angular, Vue y Svelte ofrecen diferentes alternativas para construir interfaces modernas. Por ejemplo, React trabaja con componentes, Angular proporciona una estructura más completa para construir aplicaciones y Vue busca ofrecer un enfoque progresivo basado en HTML, CSS y JavaScript.

En el backend también existen diferentes alternativas como Node.js, Express, Spring Boot, Laravel, Django, FastAPI y ASP.NET Core. No considero necesario aprender todas estas tecnologías al mismo tiempo, pero sí me parece importante conocer que existen diferentes herramientas y entender qué problema puede resolver cada una.

## **Microservicios, Docker y la nube**

Los temas de microservicios, Docker y Cloud Native me ayudaron a comprender que las aplicaciones modernas pueden tener arquitecturas mucho más complejas que una aplicación tradicional.

Los microservicios permiten dividir una aplicación en servicios pequeños y autónomos, cada uno enfocado en una capacidad específica del negocio. Una de sus ventajas es que los servicios pueden desplegarse y evolucionar de manera independiente, aunque también existen nuevos retos relacionados con la comunicación, los datos y la operación del sistema.

Por otro lado, el enfoque Cloud Native busca diseñar y operar aplicaciones aprovechando las características de la computación en la nube. En este contexto aparecen conceptos como contenedores, automatización y microservicios.

## **Git, GitHub y automatización**

El material sobre Git y GitHub también me pareció fundamental. El control de versiones permite llevar un registro de los cambios realizados en un proyecto y facilita regresar a versiones anteriores cuando sea necesario.

GitHub, además, permite trabajar colaborativamente mediante repositorios, ramas y solicitudes de cambios. Esto es especialmente importante porque el desarrollo de software normalmente no es un trabajo aislado, sino que requiere colaboración entre diferentes personas.

También me pareció interesante GitHub Actions, porque permite automatizar procesos como compilación, pruebas y despliegues mediante workflows. Esto demuestra cómo el desarrollo moderno busca reducir tareas repetitivas y hacer más eficiente el ciclo de desarrollo del software.

## Seguridad, autenticación y autorización

Considero que la seguridad es uno de los temas que no debe verse como algo adicional, sino como una parte fundamental de cualquier aplicación web.

El material permitió diferenciar autenticación y autorización. La autenticación permite comprobar quién es el usuario, mientras que la autorización determina qué puede hacer o a qué recursos puede acceder. Esta diferencia me parece sencilla, pero muy importante para comprender posteriormente sistemas de usuarios y permisos.

También pude conocer tecnologías y conceptos como JWT y OAuth. JWT permite transportar información de manera compacta y firmada, mientras que OAuth 2.0 se enfoca principalmente en la autorización y en permitir que una aplicación obtenga acceso limitado a determinados recursos.

## Inteligencia Artificial aplicada al desarrollo

La sección de Inteligencia Artificial fue una de las que más me llamó la atención porque muestra cómo está cambiando actualmente la forma de desarrollar software.

El material presenta herramientas como ChatGPT, Claude, Gemini, Microsoft Copilot y GitHub Copilot, pero también introduce conceptos que van más allá de simplemente conversar con una IA, como el uso de APIs, agentes, skills y prompt engineering.

Me parece especialmente importante aprender a utilizar estas herramientas como apoyo y no como sustituto del conocimiento del desarrollador. La Inteligencia Artificial puede ayudar a analizar problemas, explicar código, encontrar errores, generar alternativas y automatizar determinadas tareas, pero el desarrollador debe comprender, revisar y validar los resultados.

Los recursos actuales de OpenAI y Claude muestran precisamente una evolución hacia herramientas capaces de trabajar con APIs, herramientas, agentes y skills, lo que indica que la IA está pasando de ser solamente un asistente conversacional a convertirse también en una herramienta integrada dentro del proceso de desarrollo.

## Conceptos que considero más importantes

- **Arquitectura cliente-servidor:** porque permite comprender cómo se comunican el cliente y el servidor.
- **HTML, CSS y JavaScript:** porque constituyen una base fundamental del desarrollo web.
- **HTTP:** porque permite comprender cómo se realizan las comunicaciones en la Web.
- **MVC:** porque ayuda a organizar las responsabilidades dentro de una aplicación.
- **Bases de datos y SQL:** porque permiten almacenar, consultar y administrar la información.
- **JSON y XML:** porque permiten representar e intercambiar información.
- **APIs y REST:** porque facilitan la comunicación entre aplicaciones y servicios.
- **Seguridad:** porque una aplicación debe proteger tanto sus datos como a sus usuarios.
- **Git y GitHub:** porque permiten controlar versiones y trabajar colaborativamente.
- **Microservicios y nube:** porque forman parte de muchas arquitecturas modernas.
- **Inteligencia Artificial:** porque se está convirtiendo en una herramienta importante dentro del proceso de desarrollo de software.

## Conclusión

En conclusión, considero que el material de estudio fue muy útil porque permitió tener una visión general del desarrollo y la arquitectura de aplicaciones web, comenzando desde los conceptos más básicos y llegando hasta tecnologías y metodologías actuales.

Uno de los principales aprendizajes que obtengo es que desarrollar una aplicación web no consiste únicamente en crear una interfaz que pueda visualizarse en un navegador. Detrás de ella existen diferentes componentes relacionados con servidores, protocolos, APIs, bases de datos, seguridad, control de versiones, infraestructura y, cada vez más, Inteligencia Artificial.

También considero que el material deja una enseñanza importante: las tecnologías cambian constantemente, pero los fundamentos siguen siendo necesarios. Aprender HTML, CSS, JavaScript, HTTP, bases de datos, arquitectura y seguridad proporciona una base que permite posteriormente comprender y utilizar nuevas herramientas y frameworks.

Por esta razón, considero que este material no solamente fue útil para conocer diferentes tecnologías, sino también para comprender mejor el panorama actual del desarrollo de software y la necesidad de continuar aprendiendo y actualizándose como futuro profesional del área.